

텔롬엑스 MES 온톨로지 적용 설계

infurs·hisense·naturefood 세 MES가 지금 각자 다르게 부르고 있는 *같은 현실*을 하나의 객체 모델로 묶고, 그 위에 판단(Logic)·실행(Action)·권한(Security)을 얹는다. 데이터를 옮기는 작업이 아니라, 세 공장의 업무 세계를 같은 언어로 합의하는 작업이다.

문서	작성 · 개정	대상 시스템	근거	상태
TX-ONT-DESIGN r1.1	2026-09-19 · 2026-09-20	infurs_db · hisense_db · naturefood	운영 DB 실측 + 적재 결과 실측 + Palantir Ontology 설계원칙	Phase 0~3 적재 완료 (3사 286,057 트리플) · Phase 4 미착수

목차

§1	지금 왜 — 3사 실측 진단	§2	설계 원칙 7개	§3	공통 코어 온톨로지
§4	사이트별 매핑과 IRI 규칙	§5	Logic — 판단 규칙 등록부	§6	Action — 실행은 MES가 한다
§7	Security — 사람과 AI를 같은 체계로	§8	Decision Lineage	§9	배치 아키텍처와 동기화
§10	단계별 로드맵	§11	금지 목록과 성공 판정	§12	미결 사항

r1.0은 적재 전에 쓴 설계다. Phase 0~3을 실제로 돌리는 동안 설계가 틀린 곳이 네 군데 드러났고, 이 판은 그것을 반영한 것이다. 고친 결과보다 고친 근거가 오래 쓰이므로 근거를 함께 남긴다.

- **잔량 판정 기준을 문서에서 원장으로 바꿨다**(§ 3.2·§ 5). 3경로 출하가 겹치면 문서 합계가 주문수량을 넘는다 — hisense 품목 85가 문서 21 / 원장 20이었다. `tx:shippedQty` 를 신설하고 잔량·납기 이행은 이 값으로만 판정한다. 최소분(`status=9`)을 빼지 않으면 “초과출고가 있다”는 틀린 결론에 이른다.
- **클래스 14 → 16, 관계 16 → 20**(§ 3.2·§ 3.3). 출하 라인(`tx:ShipmentLine`)이 없으면 3경로를 같은 단위로 셀 수 없었고, r1.0 클래스 표에는 그림에 그려 둔 `tx:Document` 가 빠져 있었다. 관계는 `customer` · `forOrder` · `shipsTo` · `shipsLot` 이 늘었다.
- **거래처 병합 키를 “사업자번호”에서 “사업자번호 + 이름”으로 바꿨다**(§ 4.2). 한 사업자번호 아래 서로 다른 납품처가 들어 있다 — 당진시청 한 번호에 103행 (여성가족과·민원위생과...), 휴게소 운영사 한 번호에 29행이 각각 다른 지점이다. 번호로만 접으면 수주의 기관명이 엉뚱한 이름의 객체에 붙어 **납기 연락 대상을 잘못 가리킨다**.
- **적재 전 `riot --validate` 를 없앴다**(§ 9.3). 그래프 PUT이 파싱과 저장을 한 트랜잭션에서 처리하므로 검증이 중복이고, 컨테이너 안에서 JVM을 또 띄우는 것이 `moai-fuseki` 메모리 한도를 건드렸다. 적재가 5분+에서 3초로 줄었다.

트리플 규모도 추정(35만~52만)에서 실측 286,057로 교체했다 — 거래처 중복 정리가 r1.0의 예상대로 작동한 결과다(§ 9.2). 로드맵(§ 10)과 성공 판정(§ 11.2)에는 Phase 0~3의 실제 결과를, 미결 사항(§ 12)에는 해소된 항목과 새로 생긴 판단 두 건을 반영했다.

§ 01

지금 왜 — 3사 실측 진단

온톨로지를 “안 해도 되지만 있으면 좋은 것”으로 보면 이 문서는 필요 없다. 이 설계가 필요한 근거는 이 미 발생한 장애와 재작업이다. Palantir가 온톨로지 안티패턴으로 열거한 네 가지 — System Silos, Department Silos, God Object, Kitchen Sink — 가 세 MES에 각각 실물로 존재하며, 그중 하나는 2026-08-07에 실제 회귀 장애를 냈다.

안티패턴	우리 시스템의 실물	이미 치른 비용
System Silos 같은 실체를 시스템마다 따로	infurs에 품목 마스터가 둘. <code>t_item</code> 687행(수주·재고가 참조)과 <code>t_product</code> 661행 (사진·도면이 참조)이 서로 FK 없이 독립. 유일한 연결 수단이 <code>t_product.memo</code> 안의 “모델명:” 문자열.	제품사진 366건 중 품목에 붙은 것은 362건, 312품목은 끝내 미연결. 도면관리 561품목 중 335개만 사진 표시.
Department Silos 부서마다 별도 Customer	infurs 거래처 마스터도 둘. <code>t_customer</code> 18,543행, <code>t_partner</code> 9,513행. 스키마도 다름(<code>customer_type</code> vs <code>partner_type</code> ·여신·결제조건).	같은 거래처의 여신·미수·납품이력이 한 화면에 모이지 않음.
God Object 한 객체에 여러 실체	hisense <code>t_order</code> 는 28컬럼 중 9개가 거래처 실체 (<code>ordering_org</code> , <code>customer_org</code> , <code>customer_phone</code> , <code>customer_fax</code> , <code>customer_address</code> , <code>customer_contact</code> ...). 정작 <code>t_customer</code> 는 0행.	발주기관·수요기관이 문자열로만 존재 → 기관 단위 집계·명부 대조 불가.
Kitchen Sink 의미 없는 범용 슬롯	naturefood <code>mes_item</code> 48컬럼에 <code>item_param1~4</code> , <code>envelop_name1~3</code> / <code>qty</code> / <code>unit</code> , <code>clt_ctgr1~4</code> 같은 이름만 있고 의미는 코드에만 있는 필드가 절반 가까이.	필드 의미가 화면 코드에 흩어져 신규 화면마다 재해석.

안티패턴	우리 시스템의 실물	이미 치른 비용
Join Key를 관계로 착각 테이블을 그대로 객체로	<code>t_order_item</code> 의 PK 이름이 <code>item_id</code> 인데, 이것은 <i>주문 라인</i> 의 PK이고 <code>t_item.item_id</code> 는 <i>품목 마스터</i> 의 PK다. 이름만 같은 두 ID 공간.	hisense는 16개 생성지점 의미통일로 해결했다가 읽기속 12개 파일에서 회귀 장애 (2026-08-07). infurs는 같은 문제를 <code>master_item_id</code> 컬럼 추가로 회피 — 같은 문제, 다른 해법 .
공통 언어 부재	같은 "품목"이 <code>item_id</code> (infurs·hisense) / <code>it_id</code> · <code>it_no</code> (naturefood), "수주"가 <code>t_order</code> / <code>mes_sale_order</code> , "작업지시"가 <code>t_work_order</code> / <code>mes_production_work</code> .	한 사이트에서 만든 화면·규칙·점검 스크립트를 다른 사이트에 재사용 못 함.

세 MES는 데이터가 부족한 게 아니라, **같은 현실을 가리키는 이름이 세 벌인 상태**. 온톨로지가 해결하는 것은 저장이 아니라 이 이름 문제다.

1.1 이미 온톨로지가 도는 선례

맨땅이 아니다. 케어앤(caren)에서 이미 Apache Jena Fuseki를 도입해 운영 중이다 — `moai-fuseki` 컨테이너(7주 연속 가동) 안에 `/moai`와 `/caren` 두 데이터셋이 공존하고, `care-domain.ttl` 271 트리플로 질병→특기 추론(`requiresSpecialty` + `broaderSpecialty` 양방향 폐쇄)과 STT 어휘 개인화를 서비스하고 있다. SPARQL 클라이언트는 표준 라이브러리만 쓴 140행(`ontology.py`)이며, Fuseki가 죽어도 매칭·STT는 기존 경로로 조용히 풀백한다.

이 설계는 그 선례의 **운영 방식**을 그대로 물려받는다. 새 인프라를 세우는 제안이 아니라, 검증된 배치 위에 `tx-mes` 데이터셋 하나를 더하는 제안이다.

1.2 적재 후 — 진단이 실제로 해소됐는지 (r1.1)

위 표는 r1.0의 진단이다. Phase 1~3을 적재한 뒤 같은 항목을 다시 확인했다. **해소됐다는 뜻은 온톨로지 쪽에서 해소됐다는 것이고**, MES DB·화면은 그대로다(P2) — DB까지 합칠지는 여전히 판단 사항이다(§ 12 D2·D3).

안티패턴	온톨로지에서 해소된 방식 (실측)	남은 것
System Silos	infurs <code>t_item</code> 687 + <code>t_product</code> 661 → <code>tx:Item</code> 687개. <code>memo</code> 문자열 조인이 IRI 동일성으로 대체됐고 661/661 전건 매칭. 도면·제품사진 927건이 품목에 직접 붙는다.	두 마스터의 단가가 어긋난 16건(합치기 전에는 비교할 방법이 없던 값). 사진 없는 품목 324 · 도면 없는 품목 126.
Department Silos	infurs <code>t_customer</code> 18,543 + <code>t_partner</code> 9,513 = 28,056행 → <code>tx:Partner</code> 9,707객체 (65.4% 감소). 병합 근거는 <code>tx:mergedFrom</code> 에 한 행도 빠짐없이 신고, 점검이 총합을 DB 행 수와 맞춘다.	동명 기관이 여러 건이라 연결하지 않은 수주 1건. DB·화면 통합 여부(D2).
God Object	hisense <code>t_order</code> 의 인라인 기관 9컬럼 → <code>tx:Partner</code> 승격. 발주기관(조달청) 과 수요기관 이 다른 역할로 분리돼, 납기 지연 시 연락 대상이 질의로 나온다.	infurs는 <code>ordering_org</code> 가 11건 전부 NULL — 수주서 PDF 파서가 발주기관을 저장하지 않는다(D9).
Kitchen Sink	naturefood <code>item_param1~4</code> · <code>envelop_*</code> · <code>clt_ctgr1~4</code> 를 심지 않았다 — 의미를 아는 컬럼만 옮겼다(§ 11.1). infurs <code>t_customer.memo</code> 의 “거래처코드/업태/종목” 자유텍스트도 같은 이유로 제외.	제외한 컬럼의 의미를 나중에 확인하면 그때 이름을 붙여 올린다.

안티패턴	온톨로지에서 해소된 방식 (실측)	남은 것
Join Key를 관계로 착각	IRI가 클래스를 포함하므로 두 ID 공간이 섞이지 않고, “한 IRI가 두 클래스를 가짐 = 0” 불변식이 적재 시점에 잡는다. 사이트마다 반대인 관례 (infurs t_work_order.item_id =주문라인 / hisense=마스터)를 ETL에 명시했다.	DB의 컬럼명은 그대로다 — 새 SQL을 쓸 때의 위험은 남아 있다.
공통 언어 부재	hisense에서 만든 자재 영향분석 질의가 graph 이름만 바꿔 infurs에서 그대로 돌아왔다 (자재 8종이 수주라인에 걸림). 점검 스크립트도 같은 구조를 세 사이트가 공유한다.	infurs BOM이 9품목만 등록돼 있어 질의는 돌지만 답할 데이터가 적다(D8).

§ 02

설계 원칙 7개

이후 모든 설계 판단은 이 7개에서 파생된다. 원칙과 충돌하는 구현은 구현이 틀린 것으로 본다.

- P1

시스템이 아니라 현실을 모델링한다

t_order_item 을 OrderLine 으로 옮기는 게 목적이 아니다. 현장 사람이 쓰는 말 — 수주, 품목, 작업지시, 로트, 포장, 출하 — 를 먼저 정의하고 테이블을 거기에 붙인다. 테이블이 없으면 객체를 만들지 않고, 테이블이 들어어도 객체는 하나다.
- P2

MES DB가 단일 진실원천(SSOT)이다

온톨로지는 읽기 투영(read model)이다. 트리플스토어에 원본만 있는 데이터는 만들지 않는다. 온톨로지를 지우고 다시 만들어도 업무는 멈추지 않아야 한다.
- P3

IRI로 ID 공간을 물리적으로 분리한다

hisense가 치른 item_id 회귀는 두 ID 공간이 같은 이름을 썼기 때문에 생겼다. IRI는 클래스를 이름에 포함하므로 같은 실수가 구조적으로 불가능하다(§ 4.2).
- P4

시계열은 온톨로지에 넣지 않는다

naturefood mes_gathering_sensor 는 85,610행이고 계속 늘어난다. 온톨로지에는 SensorChannel 정의·임계값·설비 연결만 두고, 측정값은 SQL에 남긴다. 판단에 쓰는 것은 "가동/비가동" 같은 해석된 상태다.
- P5

Action은 온톨로지가 실행하지 않는다

쓰기는 전부 기존 MES 엔드포인트(Command)를 거친다. 이미 검증된 불변식 — 3경로 이중차감 방지, 출고 상한, 포장 내역서=실제출고 — 을 우회하는 쓰기 경로를 새로 만들지 않는다(§ 6).
- P6

자연어는 조건만 파싱하고, 실행은 화이트리스트가 한다

hisense 품목조회·naturefood 대시보드 NL검색에서 이미 채택한 원칙. LLM이 SPARQL/SQL을 생성해 그대로 실행하는 구조는 채택하지 않는다.
- P7

폴백 경로를 제거하지 않는다

Fuseki 미기동·타임아웃 시 모든 화면은 온톨로지 없이 기존 SQL 경로로 동작해야 한다. 타임아웃 상한은 caren과 동일하게 1.5s 에서 시작한다.

공통 코어 온톨로지

네임스페이스는 `http://mes.telomx.kr/ontology#` (접두어 `tx:`). 세 사이트가 클래스와 관계는 공유하고, 개체는 사이트별 named graph에 들어간다 (§ 9.2). 클래스는 16개다 — 세 MES에 실제로 테이블이 있고, 실제 질문에 쓰이는 것만 넣었다. r1.0의 14개에 적재 중 `tx:ShipmentLine` (3경로 출하를 같은 단위로 세려면 필요했다)과 `tx:Document` (그림 1에는 있었으나 표에서 빠져 있었다)를 더했다.

3.1 객체 그래프

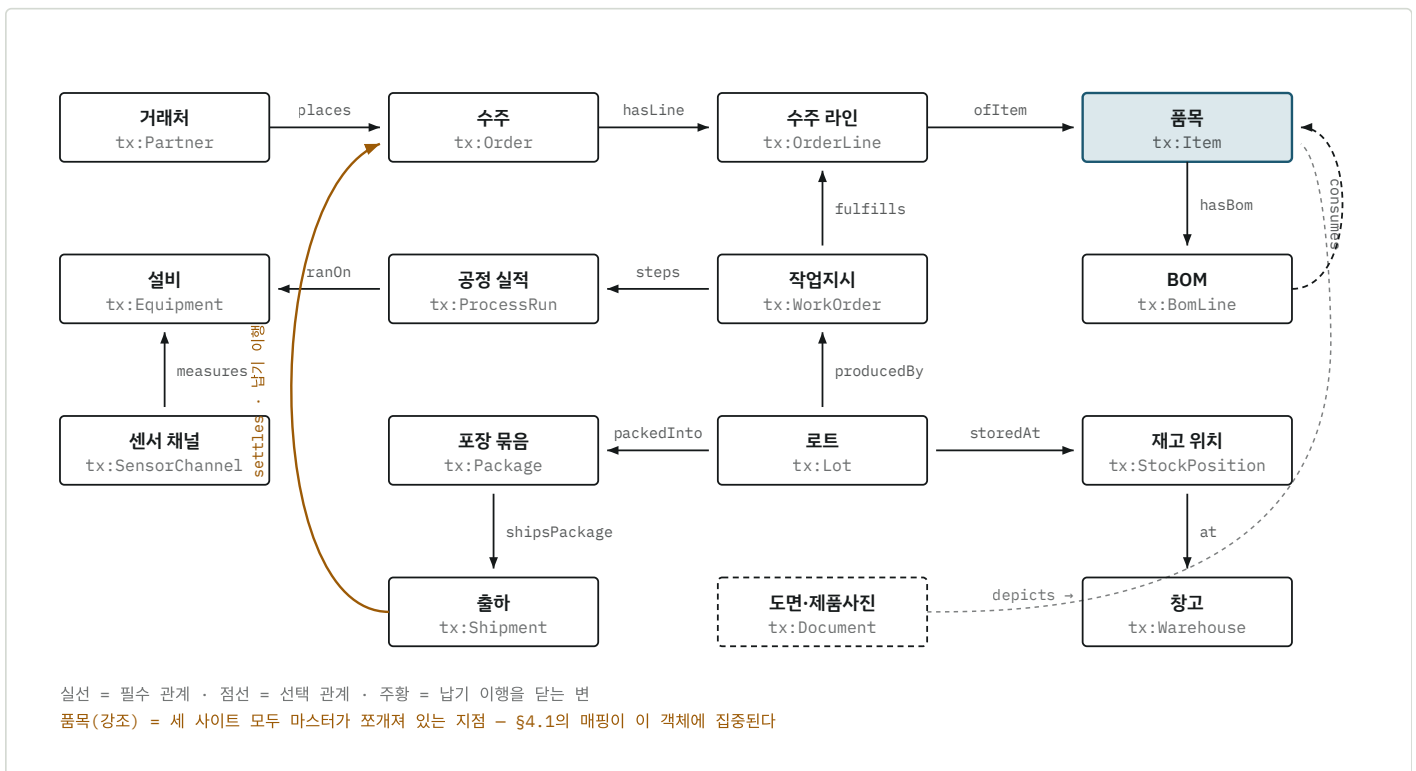


그림 1. 코어 객체 그래프. 오른쪽 위 `tx:Item`을 강조한 이유는 이 객체가 세 사이트에서 각각 다르게 쪼개져 있어서다(infurs는 `t_item` + `t_product`, hisense는 차종·ALC 속성 인라인, naturefood는 `mes_item` 48컬럼). 주황색 변 `settles`는 "이 수주가 실제로 납품됐는가"를 답하는 경로로, 현재는 수주·포장·배송지시 3경로가 따로 집계돼 누락이 났던 구간이다.

3.2 클래스 정의

클래스	현실의 무엇	핵심 PROPERTY (선별)	적재 개체수 실측 2026-09-20 · INFURS / HISENSE / NATUREFOOD
<code>tx:Partner</code>	거래처 — 발주기관·수요기관·고객사·공급사를 한 객체로	<code>bizNo</code> , <code>partnerRole</code> , <code>mergedFrom</code> , <code>tel</code> , <code>managerPhone</code>	18,205 9,707 / 4 / 8,494
<code>tx:Order</code>	수주 1건	<code>orderNo</code> , <code>orderDate</code> , <code>deliveryDeadline</code> , <code>status</code> , <code>totalAmount</code>	38 11 / 2 / 25

클래스	현실의 무엇	핵심 PROPERTY (선별)	적재 개체수 실측 2026-09-20 · INFURS / HISENSE / NATUREFOOD
tx:OrderLine	수주의 품목별 한 줄 — 잔량 판단의 기준 단위	lineNo, qty, unitPrice, deliveryDeadline, identifyNo	61 13 / 23 / 25
tx:Item	품목 — 제품·부자재·원 자재 공통	itemCode, identifyNo, itemType, unit, safetyStock, leadTimeDay	1,114 687 / 404 / 23
tx:BomLine	품목 1단 소요 구성	qty, unit, lossRate, processStep	669 49 / 600 / 20
tx:WorkOrder	작업지시	woNo, targetQty, productionDeadline, status	57 13 / 18 / 26
tx:ProcessRun	공정별 진척 실적	processStep, goodQty, defectQty, prodDate	94 44 / 18 / 32
tx:Lot	로트 — 추적성의 단위	lotNo, qty, pkgType, expiryDate, productionDate	180 — / 180 / —
tx:Package	포장 묶음 (hisense QR 라벨 단위)	packageNo, pkgType, packedAt, shippedAt	5 — / 5 / —
tx:Shipment	출하 — 배송지시·포장 출하·판매출고 3경로의 공통 상위	route(delivery/package/...), dispatchDate, deliveredAt, status	36 8 / 10 / 18
tx:ShipmentLine r1.1	출하 1건의 품목별 수량 — 3경로를 같은 단위로 세는 자리	qty(문서), shippedQty(원장), route, ofItem	76 10 / 48 / 18
tx:StockPosition	창고×품목(×로트) 현재 고	qty, updatedAt	211 25 / 171 / 15
tx:Warehouse	창고·구역	warehouseCode, name, warehouseType	7 3 / 2 / 2
tx:Equipment	설비	equipmentCode, name, model, maker	11 — / — / 11
tx:SensorChannel	센서 채널 정의 — 측정 값은 넣지 않음(P4)	channelKey, unit, runOverThreshold, scaleFactor	6 — / — / 6
tx:Document r1.1	도면·제품사진 — 그림 1에는 있었으나 r1.0 표 에서 빠져 있었다	fileType, filePath, depicts	927 927 / 0 / —

개체수는 실제 그래프를 센 값이다(2026-09-20, Fuseki tx-mes). r1.0의 추정과 달라진 곳은 둘이다. tx:Partner 는 infurs 28,056행을 9,707객체로 접어 3사 18,205가 됐고 (r1.0 추정 28,000+), tx:BomLine 은 hisense BOM 600건이 실제로 적재돼 669로 줄었다(추정 1,270). tx:Package 5· tx:Equipment 11처럼 추정보다 작은 항목은 원천이 데모 수준이라는 뜻이며, 없는 것을 만들지 않는다는 P1을 지킨 결과다. hisense tx:Partner 4는 마스터 2 + 수주에서 승격된 기관 2(조달청)다.

3.3 관계(ObjectProperty)

관계	도메인 → 레인지	이 관계가 답하는 업무 질문
tx:places	Partner → Order	이 기관이 올해 발주한 수주는?(발주기관 — hisense는 조달청)
tx:customer r1.1	Order → Partner	납기가 늦어지면 연락할 곳은?(수요기관 — 발주기관과 다르다)
tx:hasLine	Order → OrderLine Shipment → ShipmentLine	이 수주에 남은 품목별 잔량은? / 이 출하에 실린 품목별 수량은?
tx:ofItem	OrderLine · Lot · StockPosition → Item	이 품목이 걸린 수주·재고·로트 전부
tx:hasBom	Item → BomLine	이 제품을 만들려면 무엇이 필요한가?
tx:consumes	BomLine → Item	이 자재가 부족하면 못 만드는 제품은?(역방향 전이)
tx:fulfills	WorkOrder → OrderLine	이 작업지시는 어느 수주 라인을 위한 것인가?
tx:steps	WorkOrder → ProcessRun	이 지시는 어느 공정까지 왔나?
tx:ranOn	ProcessRun → Equipment	이 설비가 멈추면 지연되는 지시는?
tx:measures	SensorChannel → Equipment	이 설비의 가동 판정 근거 채널은?
tx:producedBy	Lot → WorkOrder	이 로트는 언제·어느 지시로 만들어졌나?
tx:storedAt	Lot · StockPosition → Warehouse	어느 창고에 얼마 있나?
tx:packedInto	Lot → Package	이 로트는 어느 묶음으로 나갔나?
tx:shipsPackage	Shipment → Package	이 출하에 실린 묶음
tx:shipsLot r1.1	Shipment → Lot	이 출하로 실제 나간 로트는?(출고 원장에서 역추적)
tx:forOrder r1.1	Shipment → Order	이 출하는 어느 수주의 것인가?(머리 단위 — 라인 이행은 settles)
tx:shipsTo r1.1	Shipment → Partner	이 출하를 받는 기관은?
tx:settles	Shipment → OrderLine	이 수주는 실제로 다 납품됐나? (3경로 통합 — 수량은 tx:shippedQty 로 센다)
tx:depicts	Document → Item	이 품목의 도면·제품사진
tx:sameAs	Item → Item (사이트 간)	세 공장이 같은 자재를 쓰고 있나?(Phase 4)

설계 판단

tx:consumes 와 tx:hasBom 의 **역방향 전이 폐쇄**가 이 온톨로지의 실질적 수익원이다. "이 부자재가 떨어지면 어느 수주의 납기가 위험한가"는 지금 SQL로는 BOM 단계마다 조인을 새로 써야 하지만, SPARQL에서는 `?item (^tx:consumes/^tx:hasBom)* ?product` 한 줄이다. caren의 `broaderSpecialty` 양방향 폐쇄와 같은 패턴을 그대로 쓴다.

R1.1 — 수량을 세는 속성이 둘이다

tx:qty 는 문서에 적힌 수량(배송지시·포장 내역서)이고 tx:shippedQty 는 출고 원장의 실제 수량이다. 잔량·납기 이행은 반드시 후자로 판정한다 — 3경로가 겹치면 문서 합계가 주문수량을 넘기 때문이다(hisense 품목 85: 문서 21 / 원장 20). 문서 수량도 버리지 않는다. “문서가 0”과 “문서가 아예 없다”는 다른 사실이고, 둘의 차이가 곧 **아직 출고되지 않은 지시**이기 때문이다.

사이트별 매핑과 IRI 규칙

이 표가 설계서의 실질적 산출물이다. 클래스마다 세 사이트의 실제 테이블·컬럼을 명시한다. 빈칸은 그 사이트에 해당 실체가 없다는 뜻이고, 표에 담긴 불일치가 그대로 Phase 1~3의 작업 목록이 됐다. **r1.1**에서는 적재하며 확인된 내용으로 고쳤다 — 특히 naturefood 출하 테이블과 infurs 원장 키는 r1.0의 기술이 틀렸던 지점이다.

4.1 객체 → 테이블 매핑

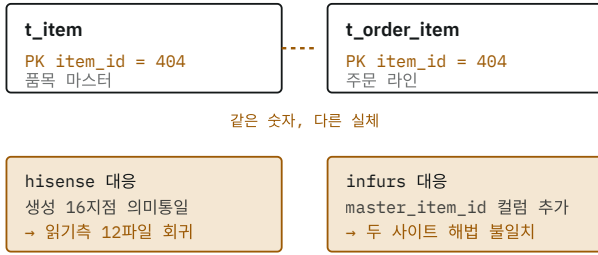
클래스	INFURS_DB	HISENSE_DB	NATUREFOOD	주의
Partner	t_customer + t_partner	t_partner (2행) + t_order 인라인	mes_client	infurs는 사업자번호+이름 으로 병합 (§ 4.2), hisense는 수주에서 승격
Order	t_order	t_order	mes_sale_order	hisense는 거래처가 이 테이블에 인라인
OrderLine	t_order_item	t_order_item	mes_sale_item	PK가 item_id — 품목 PK와 동명 (§ 4.2)
Item	t_item + t_product	t_item	mes_item	infurs는 memo 문자열로만 조인 가능 → 661/661 병합 완료 , 흔적은 tx:mergedFrom
BomLine	t_bom / t_bom_detail	t_bom / t_bom_detail	mes_production_bom	naturefood는 BOM이 작업 (work_idx)에 매달림
WorkOrder	t_work_order (master_item_id 有)	t_work_order (컬럼 無)	mes_production_plan / _work	같은 문제를 다르게 해결한 지점 — infurs item_id =주문라인, hisense=마스터. ETL이 사이트별로 갈린다
ProcessRun	t_production / t_process_step	t_production	mes_production_work / _time	naturefood는 공정 시간이 별 테이블
Lot	—	t_lot (180행)	— (바코드만, 작업지시 연결 없음)	infurs·naturefood는 로트 객체를 만들지 않는다(P1) — naturefood 바코드 연결은 Phase 4

클래스	INFURS_DB	HISENSE_DB	NATUREFOOD	주의
Package	—	t_package / t_package_item	—	hisense만 QR 라벨 단위 존재. r1.0 이 naturefood mes_ship_item 을 여기 적은 것은 오기 — 그건 출하 실적이다
StockPosition	t_stock	t_stock / t_stock_transfer	mes_warehouse_stock	naturefood는 구역 (section_name) 까지
Warehouse	t_warehouse (3)	t_warehouse (2)	mes_warehouse_in_out	
Shipment	t_delivery	t_delivery + t_package + 판매출고	mes_shipping_order (지시서)	hisense 3경로 통합이 핵심. naturefood는 지시서가 없는 옛 출하는 라인 하나짜리 출하를 만든다 — 머리 없는 라인은 집계에서 통째로 빠진다
ShipmentLine r1.1	t_delivery_item 원장 키 ref_type='delivery_item' · ref_id=t_delivery_item.id	t_delivery_item + t_package_item 원장 키 ref_id=delivery_id / package_id	mes_ship_item (실적 qty_out)	원장 키 공간이 사이트마다 다르다 — 베끼면 조용히 0건이 된다. hisense는 취소 status=9 제외 필수, infurs-naturefood에는 취소 코드가 없다
Equipment	t_facility (0행)	t_facility (0행)	mes_equipment (11행)	설비 마스터는 naturefood만 실데이터 — 나머지는 객체 0개로 적재하고 그 사실을 결함목록에 남긴다
SensorChannel	—	—	mes_gathering_sensor a1~a6	채널 정의만, 측정값 85,610행은 SQL 잔류(P4)
Document	t_drawing (561) / t_product_picture (366)	t_item_file (0행)	— (품목에 붙지 않음)	hisense는 전용 테이블 없이 공용 파일 표 재사용. 실제 문서는 infurs에만 있다(927건)

4.2 IRI 규칙 — ID 이중공간을 구조적으로 없앤다

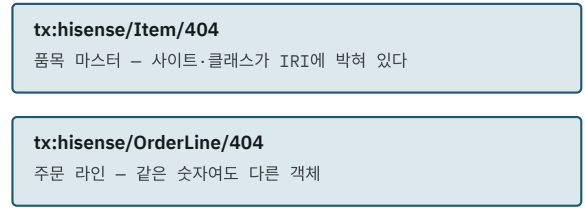
2026-08-07 hisense 장애의 원인은 **item_id** 라는 이름이 두 가지를 가리켰다는 것이다. 16개 생성 지점의 의미를 통일했더니 읽기축 12개 파일이 옛 의미를 그대로 읽어 회귀했다. IRI는 클래스와 사이트를 이름 안에 포함하므로, 두 ID가 우연히 같은 문자열이 되는 일이 구조적으로 불가능하다.

현재 - 이름이 같은 두 ID 공간



두 해법 모두 <이름이 같다>는 원인을 남겨둔 채 증상만 막았다.

IRI 도입 후 - 이름 충돌이 불가능



ETL이 IRI를 만들 때 클래스를 반드시 지정하므로, 잘못 매핑하면 적재 단계에서 형(型) 검증에 걸린다 - 런타임 오조인이 아니라 빌드 오류가 된다.

그림 2. 같은 결함에 대한 두 사이트의 대응과, IRI가 그 결함을 없애는 방식. 핵심은 "온톨로지가 더 똑똑해서"가 아니라 **오류 발생 시점이 런타임에서 적재 시점으로 옮겨진다는 점**이다.

IRI 생성 규칙

`http://mes.telomx.kr/id/{site}/{Class}/{naturalKey}`

- `{site}` - infurs | hisense | naturefood. 사이트를 IRI에 넣어 named graph와 이중 방어.
- `{Class}` - § 3.2 클래스명 그대로. 이 조각이 ID 공간을 분리한다.
- `{naturalKey}` - PK가 아니라 업무 키를 우선한다. Order 는 order_no, Item 은 item_code, Lot 은 lot_no. PK는 업무 키가 없을 때만 쓰고 그 경우 pk- 접두어를 붙인다 (OrderLine/pk-23).
- 업무 키를 쓰는 이유: 전체 재적재(full reload) 후에도 IRI가 안 바뀌어 외부에서 인용한 링크가 유지된다.

r1.1 - 적재하며 추가된 규칙

업무 키를 쓰려면 그 키가 정말 **유일한지**를 적재가 확인해야 한다. 아래 넷은 실제로 안 유일했던 사례에서 나왔다.

- 자연키가 비었거나 중복이면 **pk-** 로 떨어뜨리고 결함으로 기록한다. 조용히 넘어가면 서로 다른 두 행이 한 객체로 합쳐진다 - 발견하기 가장 어려운 종류의 사고다.
- infurs **WorkOrder** 는 처음부터 PK를 쓴다. wo_no 가 수주 단위로 발급돼 라인마다 중복이다 (W0-260727-004 가 2행). 자연키로 쓰면 두 작업지시가 한 객체가 된다.
- Partner** 의 자연키는 “사업자번호 + 이름”이다 - Partner/biz-3118301218-당진시청 형태. 번호만 쓰면 한 법인의 부서·지점이 뭉개지고(당진시청 103행), 이름만 쓰면 동명의 다른 사업체가 합쳐진다(“개별화물”이 서로 다른 14곳). 실측 92개 법인이 부서·지점을 갖고 있고, 그것들은 tx:bizNo 가 같아 질의로 묶인다.
- 모든 객체는 tx:sourceTable · tx:sourcePk 를 신고, 병합된 객체는 tx:mergedFrom 을 원천 행마다 하나씩 신는다. 병합은 판단이고, 되짚을 수 없는 판단은 쓸 수 없다. 점검이 mergedFrom 총합을 DB 행 수와 맞춰 병합 중 조용히 사라진 행을 잡는다(infurs 28,056 = 28,056).

Logic — 판단 규칙 등록부

판단 규칙은 이미 전부 존재한다. 문제는 그것이 *PHP 컨트롤러 안에 흩어져* 있어서 같은 규칙이 사이트마다 다르게 구현되고, 어느 화면이 어느 규칙을 쓰는지 아무도 모른다는 것이다. Logic 계층의 목적은 규칙을 새로 만드는 게 아니라 등록하는 것이다 — 온톨로지에는 규칙의 식별자·입력 객체·임계값·구현 위치를 두고, 계산 자체는 기존 코드가 한다.

규칙	입력 객체	현재 구현 위치	등록 형태
안전재고 미달 판정	StockPosition, Item	Inventory.php · t_item.safety_stock / reorder_point	임계값을 tx:Item Property로 승격
BOM 소요량 전개	Item, BomLine, WorkOrder	Bom.php, WorkOrder.php	전이 폐쇄 SPARQL + 수량은 SQL
수주 잔량 / 배송대상 판정	OrderLine, Shipment, ShipmentLine	hisense 3경로 합산(Dashboard.php, Delivery.php, Package_model::shipped_out_qty())	구현 완료 — tx:settles 경유 단일 질의. 수량은 tx:shippedQty (원장)로 세며 문서 수량과 분리한다
납기 위험	Order, OrderLine, WorkOrder	사이트별 산재	Function 등록 + 위험도 산식 단일화
설비 가동 판정	SensorChannel, Equipment	naturefood Monitor.php 임계값	tx:runThreshold Property (SSOT)
포장 속도 / 리드타임 KPI	ProcessRun, Package	naturefood 공정모니터링 — 640 스틱/분, 포장구간=실적창	KPI 정의를 온톨로지에, 값은 SQL 집계
KPI 목표·실적	전 객체	hisense t_kpi_config (6행) / t_kpi	KPI ↔ 객체 연결을 Link로 명시
품목 자동분류	Item	hisense 조달청 XLS 파싱 item_type 자동분류	분류 규칙을 클래스 계층으로 표현

R1.1 — 이 규칙만은 온톨로지가 판정 기준을 바꿨다

출고 잔량의 기준을 문서에서 원장으로 바꾼 것은 r1.1에서 유일하게 *업무 규칙 자체*가 확정된 대목이다. 기존 화면들은 배송지시 문서 수량으로 잔량을 세고 있었고, 3경로가 겹치는 수주에서 그 합계가 주문수량을 넘었다 (hisense 품목 85: 문서 21 / 원장 20). 원장(t_stock_transaction)이 사실이므로 그것으로 판정한다.

여기엔 함정이 둘 있고 둘 다 실제로 밟았다. ① 취소분(hisense status=9)을 빼지 않으면 “초과출고가 있다”는 틀린 결론이 난다 — 포장 출하 → 취소 → 재출하의 자취였고, 제대로 세면 19+1=20으로 주문수량과 정확히 맞았다. ② ETL이 문서를 따라가며 원장을 붙이므로 **문서 없는 원장 출고는 조용히 빠진다**. 그래서 점검에 “원장 실출고 = 그래프 출하수량”을 넣어 반대쪽을 센다 (hisense 183=183, infurs 72=72, naturefood 23,002=23,002).

규칙을 온톨로지로 옮기지 않는다. 옮기면 SSOT가 둘이 되고(P2 위반), 기존 화면이 Fuseki 가동에 의존하게 된다(P7 위반). 온톨로지는 "이 규칙이 존재하고, 임계값은 이것이고, 구현은 여기 있다"를 아는 목록이다. 단, **임계값** 숫자는 예외로 온톨로지를 SSOT로 삼는다 – 지금 세 사이트에 흩어진 임계값이 서로 다른 게 실제 문제이기 때문이다.

Action — 실행은 MES가 한다

블로그가 강조한 "Action Loop를 닫는다"는 부분은 우리 환경에서 가장 조심해야 하는 대목이다. 팔란티어는 Foundry가 원천이라 온톨로지가 직접 write-back한다. 우리는 반대로 **MES DB가 원천**이고, 그 DB에는 이미 피로 배운 불변식이 걸려 있다.

이미 존재하는 불변식 — 우회하면 안 되는 것들

- **3경로 이중차감 방지** — 판매출고가 `sales_order` / `package` / `delivery` 세 경로로 발생하며, 같은 수량이 두 번 차감되면 안 된다.
- **출고 상한** — 배정 로트 분기가 상한을 적용하지 않아 초과출고가 났던 이력. 상한은 품목별 잔량 기준.
- **포장 내역서 = 실제 출고** — 이 등식은 확정된 불변식이다.
- **작업지시 생성은 공용 헬퍼만** — `resolve_master_item_id()` / `link_work_order_to_plan()` . 16지점 일원화 결과를 되돌리는 경로를 만들지 않는다.
- **마이그레이션 실패 시 코드까지 롤백되는 배포 스크립트가 있다** — 스키마 변경은 최소화한다.

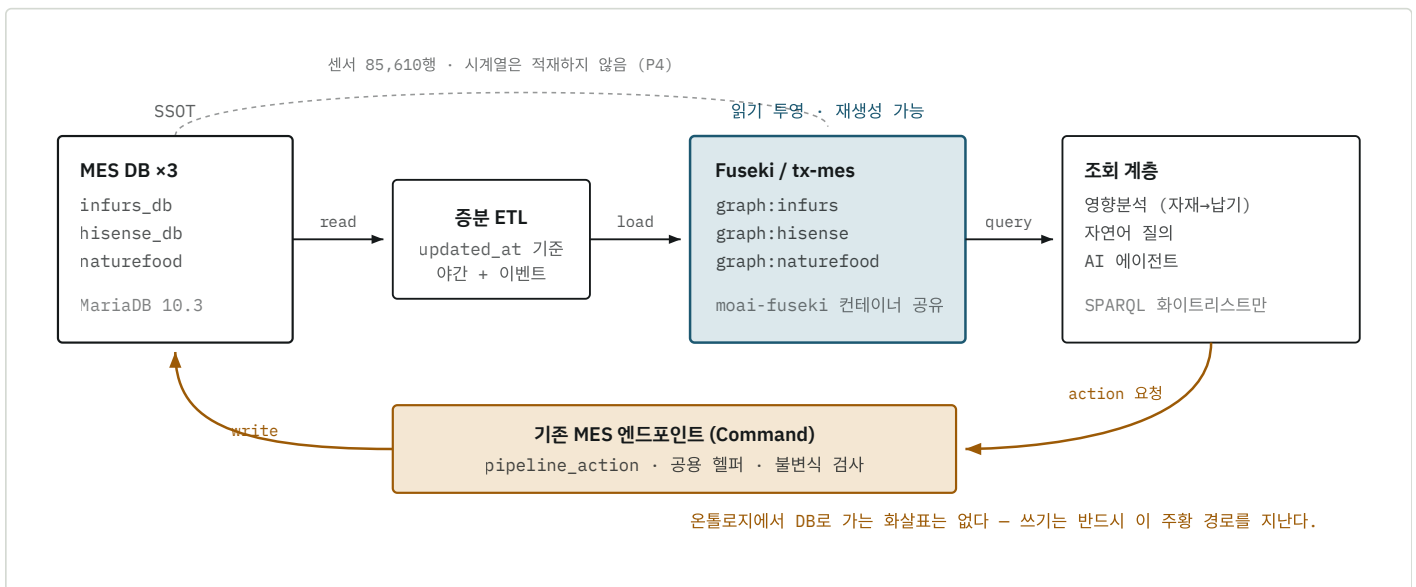


그림 3. 배치와 Action 루프. 온톨로지는 조외 계층에만 답하고, 상태를 바꾸는 요청은 전부 기존 MES 엔드포인트를 거쳐 불변식 검사를 받는다. 이 구조면 Fuseki가 죽어도 업무가 멈추지 않고(P7), 온톨로지를 전체 재적재해도 데이터가 유실되지 않는다(P2).

Action 정의 형식

온톨로지에 등록하는 Action은 "무엇을 할 수 있는가"의 선언이며, 실행 주체는 항상 MES다. 각 Action은 ① 대상 클래스 ② 호출할 엔드포인트 ③ 필요 권한 ④ 검증할 불변식 네 가지를 갖는다. 1차 등록 대

상은 이미 화면이 있는 것들로 한정한다 — 작업지시 발행, 공정 진척 기록, 포장 묶음 생성, 출하 확정, 재고 이동, 배송완료 처리.

§ 07

Security — 사람과 AI를 같은 체계로

AI가 조화·실행을 하기 시작하면 권한은 부가기능이 아니라 구조가 된다. 다행히 우리에게는 이미 권한 테이블이 있다 — `t_user` / `t_permission` / `t_menu` (infurs·hisense), `mes_menu_users` 430행(naturefood). Action의 권한은 새로 정의하지 않고 이 메뉴 권한에 매핑한다.

- Fuseki는 localhost 바인딩을 유지한다. `moai-fuseki` 는 host network + `--localhost` 로 떠 있다. 외부 노출 금지 — 웹에서 SPARQL 엔드포인트를 직접 부르는 구조를 만들지 않는다.
- 질의는 서버측 화이트리스트만. 파라미터만 바인딩하는 고정 SPARQL 템플릿. 자연어는 조건 파싱에만 쓴다(P6).
- 행 단위 제한은 named graph로. 사용자가 속한 사이트의 graph만 질의 대상에 포함한다. 사이트 간 조화는 Phase 4에서 별도 권한으로.
- AI 에이전트의 도구 범위 = 그 계정의 메뉴 권한. 사람이 못 하는 일을 에이전트가 할 수 있는 경로를 만들지 않는다.
- 선결 조치 r1.1 부분 해소. `login.php` 에 admin 비밀번호가 평문으로 박혀 있던 문제는 hisense는 코드 제거·비밀번호 교체까지 완료됐다(2026-09-20 확인). infurs와 ssfc는 그대로 남아 있다(같은 원본에서 갈라진 세 사이트의 공통 결함). Phase 1~3 적재는 읽기 전용이라 이 조건과 무관하게 진행했지만, infurs에 조화 화면이나 Action을 붙이기 전에는 선결 조건이다 — 권한 체계 위에 AI를 올리는 설계에서 최초 관문이 뚫려 있으면 나머지 설계가 무의미하다. 코드 수정과 비밀번호 변경을 함께 해야 한다 (§ 12 D7).

§ 08

Decision Lineage

"데이터가 어디서 왔나"(Data Lineage)보다 넓은 "누가 무엇을 보고 어떤 판단을 했고 그 결과가 어땠나"를 기록하자는 개념이다. 우리는 이미 조각을 갖고 있다 — 다만 서로 연결되지 않은 로그들이다.

기존 로그	담고 있는 것	LINEAGE로 승격시 추가할 것
t_audit_log	user_id, action, target, ref_id, summary, ip	대상을 IRI로 기록 — 테이블명+PK 대신
t_ai_parse_log	OCR 텍스트, AI 결과, 정규식 결과, parse_method	어느 Logic·모델 버전이 쓰였는지
t_ocr_correction	사람이 고친 값	판단 주체(사람/AI)와 수정 근거 — 재학습 입력
ai_rule_history	규칙 변경 이력 (현재 0행)	임계값 변경 → 영향받은 판단 연결
t_document_log	문서 발행 이력 17행	어느 Order-Shipment IRI에 귀속되는지

새 로그 테이블을 만들지 않는다. `t_audit_log.target` 에 들어가는 문자열을 IRI 형식으로 바꾸는 것만으로 다섯 로그가 같은 객체를 가리키게 된다. MariaDB 10.3에서 `AFTER` · `INSTANT` 컬럼 추가가 지원되지 않는 제약도 이 방식이면 피할 수 있다.

배치 아키텍처와 동기화

9.1 서버 제약이 설계를 결정한다

이 서버는 RAM 3,665MB에 가용 1,320MB다. 새벽 백업의 git bundle이 반복 OOM kill을 당한 이력이 있고, Claude 세션에는 cgroup 캡이 걸려 있다. 따라서 **새 컨테이너·새 JVM을 띄우지 않는다**. caren과 동일하게 이미 7주 가동 중인 `moai-fuseki` 에 `tx-mes` TDB2 데이터셋 하나만 추가한다.

9.2 트리플 규모 — 추정과 실측

NAMED GRAPH	적재 객체수 (실측)	객체당 트리플	적재 트리플 (실측)	R1.0 추정
graph:infurs	11,497	15.7	180,337	245K ~ 368K
graph:hisense	1,485	11.9	17,604	24K ~ 34K
graph:naturefood	8,715	10.1	87,672	78K ~ 117K
graph:schema	클래스 16 · 관계 20 · 속성 98	—	444	~400
합계 (세션·로그·시계열 제외)			286,057	35만 ~ 52만

추정보다 작아진 이유는 하나다 — `graph:infurs` 의 대부분을 차지하던 거래처 중복이 28,056행에서 9,707객체로 접혔다(r1.0이 “Partner 정리로 절반 이하”라고 적은 그 예상이 맞았다). 반대로 **객체당 트리플은 추정(8~12)보다 많다**(infurs 15.7). 병합 근거(`tx:mergedFrom`)를 원천 행마다 하나씩 싣기 때문이며, 되짚을 수 있는 병합을 위해 의도적으로 치르는 비용이다. hisense가 추정 하단보다 작은 것(17.6K)은 `t_item_file` · `t_company` 처럼 비어 있는 원천에 객체를 만들지 않았기 때문이다(P1). naturefood 센서 85,610행을 넣으면 규모가 이 표와 무의미해진다 — P4를 지키는 이유다. TDB2에서 28만 트리플은 caren(271 트리플)과 인스턴스를 공유해도 문제없고, 실측 질의 응답은 10ms 수준이다.

9.3 동기화 방식 (r1.1 — 실제 운영 값으로 교체)

- 야간이 아니라 **매시 전체 재적재**. 실측 적재 시간이 hisense 3초 · naturefood 6초 · infurs 14초라 야간까지 기다릴 이유가 없었다. cron은 `:25 / :40 / :55` 로 나눠 서로, 그리고 백업(OOM 이력)과 겹치지 않게 뒀다. **중분 ETL은 만들지 않았다** — 전체 재적재가 이 속도면 중분 로직의 버그 위험을 살 이유가 없다(2차 계획은 보류).
- 그래프 단위 **PUT이 트랜잭션이다**. 적재는 `PUT /data?graph=...` 하나로 통째 교체된다. 실패하면 아무것도 바뀌지 않으므로 이전 그래프가 그대로 남는다(부분 적재 없음) — 실측 확인.
- 적재 전 **riot --validate** 는 없었다. PUT이 파싱과 저장을 한 트랜잭션에서 하므로 깨진 입력은 400(줄·칸 번호 포함)으로 거부되고 그래프는 보존된다. 검증용 컨테이너 안에서 JVM으로 또 돌리던

것이 `moai-fuseki` 메모리 한도에 붙어 스래싱했다 — 다시 넣지 말 것(적재 5분+ → 3초).

- **겹침 방지와 상태 노출.** cron 진입점은 `flock` 으로 겹침을 막고, 한 줄 로그와 `out/status.json` (`data_at` = *마지막으로 성공한* 적재 시각)을 남긴다. 실패한 실행이 이 값을 밀어 올리면 화면이 “방금 기준”이라고 거짓말한다 — 그래프는 옛것 그대로인데, `hisense /impact` 는 이 값을 **데이터 기준 시각** 칩으로 띄우고 2회분(2시간 30분)을 넘기면 경고한다.
- **불변식 점검.** 적재 직후 자동 실행되며 실패하면 종료코드 1이다. 항목 수는 `hisense 25 · naturefood 25 · infurs 34`. 세 사이트가 같은 구조를 공유하되 사이트 고유 불변식을 더한다 — `infurs`는 “병합 근거 총합 = DB 행 수”, 공통으로 “한 IRI가 두 클래스를 가짐 = 0” · “스키마에 없는 술어 = 0” · “원장 실출고 = 그래프 출하수량”.
- **외부 의존 없음.** 이 서버는 `npm·jsdelivr·Docker Hub·Cloudflare` 대역이 차단돼 있다. ETL은 서버에 이미 있는 PHP CLI 또는 표준 라이브러리 Python으로만 만든다(`caren ontology.py` 가 `urllib` 만 쓴 이유).

§ 10

단계별 로드맵

각 단계는 **그 단계만으로 눈에 보이는 성과가 있어야 한다**. “온톨로지를 다 만든 다음에 효과가 난다”는 계획은 중간에 멈추면 전부 손실이므로 채택하지 않는다.

r1.1 기준 진행 상황: Phase 0~3 완료(2026-09-19 ~ 09-20), Phase 4 미착수. 각 단계에 **완료 표시**와 **실제 결과**를 덧붙였다 — 계획과 결과가 다른 곳이 다음 단계의 근거가 된다.

Phase 0

합의

✓ 완료
2026-09-19

용어 합의와 스키마 확정

- § 3.2 클래스 14개 · § 3.3 관계 16개를 현장 용어로 검토 — 세 사이트 담당자가 같은 말을 쓰는지 확인
- `tx:` 네임스페이스·IRI 규칙 (§ 4.2) 확정, `mes-core.ttl` 작성 후 `riot --validate`
- Fuseki에 `tx-mes` 데이터셋 생성 + `graph:schema` 적재

산출물 `mes-core.ttl` · 용어 대조표 · 빈 데이터셋 · 검증 문법 검증 통과, 트리플 ~400

결과 `mes-core.ttl` (현재 r1.2: 클래스 16 · 관계 20 · 속성 98, 444 트리플), `tx-mes` 데이터셋 생성. **세 사이트가 이 파일 한 벌을 공유하고 복사본을 두지 않는다** — 원본은 `hisense` 레포에 있고 나머지 사이트는 경로로 읽는다. 사이트가 멋대로 만든 술어는 “스키마에 없는 술어 = 0” 불변식이 잡는다. **용어 합의 회의는 아직 못 했다** — 클래스·관계는 DB 실측과 기존 화면 용어로 정했다.

Phase 1

적재

✓ 완료
2026-09-19

한 사이트 전체 적재 — `hisense` 먼저

- `hisense`를 1번으로 두는 이유: 객체 2,417행으로 가장 작고, Lot·Package·3경로 출하가 *유일하게* 다 있는 사이트라 모델 검증이 제대로 된다
- ETL 작성(PHP CLI) → `graph:hisense` 적재 → 객체 수·필수 Link 누락 점검
- 부수 성과: 적재가 실패하는 행이 곧 데이터 결함 목록이 된다(고아 명세서·미연결 도면 등)

산출물 ETL 스크립트 · 적재 리포트 · 결함 목록 · 검증 2.4만~3.4만 트리플, 필수 Link 누락 0

결과 `graph:hisense` **17,604 트리플**(D1대로 `hisense` 먼저), 점검 25항목 통과 · 필수 Link 누락 0. **모델이 실제로 고쳐진 단계다** — `tx:ShipmentLine` · `tx:shipsLot` 신설, 그리고 **출하 수량을 문서가 아니라 원장으로** 판정하도록 바꿨다 (§ 5). 예고했던 “결함 목록”은 실제로 나왔다: 문서≠원장 6건 · 품목 없는 출하 1 · 거래처 객체 없는 수주 2.

Phase 2

첫 수익

✓ 완료
2026-09-19

자재 영향분석 화면 — 온톨로지가 아니면 못 만드는 것

- 질문: "이 부자재가 떨어지면 어느 수주의 납기가 위험한가?" — `^tx:consumes/^tx:hasBom` 전이 폐쇄
- 현재 SQL로는 BOM 단계마다 조인을 새로 써야 해서 화면이 없다. SPARQL 한 질의로 대체
- `tx:settles` 로 3경로 출하를 합산해 수주 잔량을 단일 질의로 — 3차 스왑에서 손으로 맞춘 로직의 겸 산에도 쓴다
- Fuseki 미기동 시 "영향분석 일시 사용 불가" 표시 후 기존 화면 정상 동작(P7)

산출물 영향분석 화면 1개 · 화이트리스트 SPARQL 템플릿 · 검증 기존 수주잔량 집계와 수치 일치

결과 hisense `/impact` 화면(재고 메뉴 하위), 화이트리스트 SPARQL 4개, 실측 응답 ~10ms(폴백 타임아웃 1.5s). `/impact/verify` 가 온톨로지 잔량과 `Package_model::shipped_out_qty()` 를 대조해 **23라인 전부 일치**. Fuseki를 세우면 자재목록(SQL)은 그대로 쓰고 분석 영역만 물러난다 — P7 실측 확인. 이 단계에서 **ETL 회귀를 하나 잡았다**: 포장 경로 출하라인을 머리에 잇는 줄이 빠져 포장 QR 출고본이 집계에서 통째로 누락됐다. 객체 수 점검은 통과했고 **연결의 반대쪽을 세지 않아서** 못 잡은 것이라, 점검에 “출하에 안 붙은 출하라인 = 0”을 추가했다.

Phase 3

확산

✓ 완료
2026-09-20

infurs·naturefood 적재와 Partner 정리

- infurs: `t_item` + `t_product` 를 하나의 `tx:Item` 으로 — `memo` 문자열 조인을 IRI 동일성으로 대체. 미연결 312품목이 여기서 드러난다
- infurs: `t_customer` (18,543) + `t_partner` (9,513) 중복 정리 → `tx:Partner` 실제 규모 확정
- naturefood: `mes_item` 48컬럼 중 실제 쓰이는 Property만 선별(Kitchen Sink 해소), `SensorChannel` 6채널 정의·임계값 등록
- hisense `t_order` 의 인라인 거래처 9컬럼 → `tx:Partner` 객체로 승격(DB는 그대로, 온톨로지에 서 분리)

산출물 3그래프 적재 완료 · 중복 정리 리포트 · 검증 35만~52만 트리플, 사이트별 불변식 점검 통과

결과 3그래프 **286,057 트리플**(추정보다 작은 이유는 § 9.2), 사이트별 점검 전향목 통과(hisense 25 · naturefood 25 · infurs 34). 거래처 중복 정리: infurs 28,056행 → **9,707객체(65.4% 감소)**, 리포트 9,688행. 품목 두 마스터는 661/661 병합 — **예고했던 “미연결 312품목”은 실제로는 사진이 없는 품목 324건**이었고, 마스터 매칭 자체는 전건 성공했다. hisense 인라인 거래처는 발주기관(조달청)·수요기관으로 분리 승격. naturefood는 Kitchen Sink 컬럼을 승계하지 않고 센서는 채널 6개의 정의·임계값만 실었다. **계획과 달랐던 것**: 거래처 병합 키를 사업자번호만으로 잡으면 부서·지점이 뭉개져 “번호+이름”으로 바꿨다(§ 4.2).

Phase 4

운영

미착수

Action 등록 · Lineage · 자연어 질의

- Action 6종 등록(§ 6) — 대상 클래스·엔드포인트·권한·불변식 4요소
- `t_audit_log.target` 을 IRI로 전환 → 5개 로그가 같은 객체를 가리킴(§ 8)
- 기존 NL검색(hisense 품목조회·naturefood 대시보드)의 조건 파싱 결과를 SPARQL 화이트리스트로 라우팅
- `tx:sameAs` 로 사이트 간 동일 자재 연결 — 세 공장 공통 구매·재고 재배분 검토의 전제

산출물 Action 등록부 · Lineage 전환 · NL 질의 연결 · 검증 권한 밖 Action 차단 확인, 감사 로그로 판단 추적 가능

착수 전에 정리할 것(r1.1) ① infurs BOM 등록 — 질의는 그래프 이름만 바뀌 돌지만 등록된 BOM 9품목이 실제 수주와 거의 겹치지 않아 **영향분석 화면을 붙여도 답할 데이터가 없다**(D8). ② infurs-ssfc `login.php` 평문 비밀번호(§ 7·D7). ③ LLM 크레딧(D6). ④ naturefood 로트-작업지시 연결(현재 바코드만).

Phase 4의 자연어 질의는 LLM 크레딧을 쓴다. hisense 자연어검색·일반수주서 OCR이 이미 **크레딧 부족으로 라이브 미검증** 상태다. Phase 4 착수 전에 크레딧 확보 여부를 먼저 확인해야 하며, 확보 안 되면 Phase 4에서 자연어 항목만 분리해 뒤로 미룬다 — Action 등록과 Lineage는 크레딧과 무관하다.

§ 11

금지 목록과 성공 판정

11.1 하지 않는 것

- 테이블을 그대로 객체로 옮기지 않는다. `t_order_item` 한 행에 품목명·규격·단가가 복사돼 있어도, 객체는 `OrderLine` 과 `Item` 둘로 나눈다.
- 사이트별로 `Item` 클래스를 따로 만들지 않는다. 그것이 System Silos의 재산이다. 차이는 Property로, 클래스로는 만들지 않는다.
- 쓰이지 않는 컬럼을 Property로 올리지 않는다. naturefood `item_param1~4` 처럼 의미가 코드에만 있는 필드는 의미를 확인한 것만 이름 붙여 올린다.
- 온톨로지에만 있는 데이터를 만들지 않는다. 예외는 임계값(§ 5)뿐이며, 그것도 DB에 반영 경로를 함께 만든다.
- 기존 화면을 Fuseki 의존으로 바꾸지 않는다. 온톨로지는 새 화면을 추가할 뿐이다.
- LLM이 생성한 질의를 그대로 실행하지 않는다.
- 새 Fuseki 인스턴스·새 컨테이너를 띄우지 않는다. RAM 제약(§ 9.1).
- 적재 과정에서 JVM을 따로 띄우지 않는다. r1.1 추가 `riot --validate` 를 컨테이너 안에서 돌리던 것이 메모리 한도에 붙어 스래싱했다(§ 9.3). 검증은 PUT이 트랜잭션으로 한다.
- 병합한 객체에서 원천 행을 지우지 않는다. r1.1 추가 두 마스터를 한 객체로 합칠 때 `tx:mergedFrom` 으로 원천 행을 전부 남긴다. 되짚을 수 없는 병합은 검증도 못 하고 틀렸을 때 고칠 수도 없다(§ 4.2).

11.2 성공 판정

다음 네 가지가 되면 이 설계는 성공한 것으로 본다. 되지 않으면 트리플 수와 무관하게 실패다. r1.1에서 Phase 0~3 기준의 판정 결과를 덧붙였다.

판정 항목	판정 기준	R1.1 결과 (2026-09-20)
영향 추적	"이 자재가 없으면 위험한 수주"를 한 질의로 답한다 — 지금은 답하는 화면이 없다	달성(hisense) — <code>/impact</code> 화면, 전이 폐쇄 한 질의, ~10ms. infurs는 질의는 돌지만 BOM 데이터가 없어 답이 비어 있다(D8). naturefood는 화면 미구축.
ID 사고 재발 방지	품목 ID와 주문라인 ID를 혼용한 코드가 적재 단계에서 걸린다 — 지금은 런타임 회귀로 발견된다	달성 — “한 IRI가 두 클래스를 가짐 = 0” · “OrderLine IRI에 <code>pk-</code> 접두어”가 세 사이트 점검에 상시 포함. 다만 적재 점검은 ETL만 지킨다 — DB에 새 SQL을 쓰는 코드는 여전히 사람이 조심해야 한다.
재사용	한 사이트에서 만든 질의·점검이 graph 이름만 바뀌 다른 사이트에 돈다	달성 — hisense 영향분석 질의를 <code>graph</code> 이름만 바꿔 infurs에서 실행(자재 8종이 수주라인에 걸림). 점검 스크립트도 세 사이트가 같은 구조를 공유한다.
무해성	Fuseki를 정지시켜도 세 MES의 기존 화면이 전부 정상 동작한다	달성 — 온톨로지를 읽는 화면은 hisense <code>/impact</code> 하나뿐이고 그마저 폴백한다(실측). 나머지 화면은 Fuseki를 모른다. ETL은 읽기 전용이라 MES DB에 쓰지 않는다.

네 항목 모두 hisense에서는 달성, 세 사이트 전부는 미완이다. 남은 것은 설계 문제가 아니라 데이터 공백(infurs BOM)과 화면 부재이며, 그래서 Phase 4 착수 전 정리 목록에 넣었다.

§ 12

미결 사항

설계를 확정하려면 판단이 필요한 항목들이다. 전부 사용자 결정 사항이다. r1.1에서 해소된 항목은 그대로 두고 상태만 적었다 — 무엇이 왜 그렇게 정해졌는지가 다음 판단의 근거이기 때문이다.

#	미결 사항	결정이 필요한 이유 · R1.1 상태
D1	Phase 1 대상이 hisense가 맞는지	해소 — hisense로 진행했고 모델 검증이 실제로 거기서 이뤄졌다 (Lot·Package·3경로가 다 있는 유일한 사이트라 tx:ShipmentLine ·원장 기준 판정이 이때 나왔다). 이후 naturefood → infurs 순서로 확산.
D2	infurs 거래처 이중 마스터를 DB에서도 통합할지	여전히 미결 . 온톨로지에서는 28,056행 → 9,707객체로 합쳐졌고 병합 규칙·근거·리포트가 다 있다 (§ 4.2). DB 통합은 그 규칙을 그대로 쓸 수 있지만 화면·여신·결제조건 영향 범위가 크다. 결정에 쓸 재료가 이제 있다 는 점만 달라졌다.
D3	tx_item + tx_product 통합의 최종 방향	여전히 미결 . 온톨로지 동일성으로는 661/661 합쳐졌다. 부수로 두 마스터의 단가가 어긋난 16건이 드러났으니 DB를 합치든 안 합치든 이 16건은 정리해야 한다.
D4	세 사이트 공통 자재 연결 (tx:sameAs)을 어디까지 볼지	공장 간 재고 재배분은 영업·구매 정책 문제여서 기술 판단만으로는 못 정한다. Phase 4 항목.
D5	임계값 SSOT를 온톨로지로 옮기는 범위	안전재고·가동판정·KPI 목표 중 어디까지를 온톨로지가 소유할지. 현재는 naturefood 센서 임계값만 그래프에 있고 SSOT는 여전히 GatheringSensorModel.php 다 — 그 파일이 바뀌면 ETL 표도 함께 고쳐야 하는 이중 관리 상태다.
D6	Phase 4 자연어 항목의 LLM 크레딧	미확보 시 해당 항목만 분리 연기 필요 (§ 10 전제 조건). Action 등록·Lineage는 크레딧과 무관하다.
D7	login.php 평문 admin 비밀번호 선결 조치	hisense 해소 (코드 제거 + 비밀번호 교체, 2026-09-20 확인) · infurs-ssfc 미조치 . infurs에 조회 화면을 붙이기 전 선결 조건 (§ 7). 데모·현장 시연이 자동입력에 의존할 수 있어 교체 시점을 정해야 한다.
D8	infurs BOM 등록 r1.1 신규	BOM이 등록된 9품목이 실제 수주 품목과 거의 겹치지 않아(IFS-NV06 · IFS-NL03 둘뿐) 영향분석 화면을 붙여도 답이 비어 있다. 온톨로지가 아니라 기준정보의 공백 이므로 누가 언제 BOM을 등록할지가 결정 사항이다.
D9	infurs 발주기관 저장 r1.1 신규	tx_order.ordering_org 가 11건 전부 NULL이다 — 수주서 PDF 파서가 발주기관을 저장하지 않는다. hisense처럼 발주기관(조달청)과 수요기관을 나눠 보려면 파서를 고쳐야 하고, 그게 필요한 업무인지부터 확인이 필요하다.